AI-Powered Messaging Platform

Overview

Modern messaging platforms must support real-time communication at massive scale while ensuring low latency, guaranteed message delivery, and operational resilience. Users expect seamless 1:1 and group messaging, media sharing, message synchronisation across devices, and intelligent conversational experiences. Organisations also increasingly require AI-powered search, conversation summarisation, and contextual knowledge retrieval from historical conversations. Build an AI-powered messaging platform that balances large-scale distributed systems design with Gen AI, NLP, and Agentic AI-driven conversational intelligence.

Sample queries your system should handle:

- Find that message where Priya shared the project deadline last month — semantic search across chat history.
- Summarise everything discussed in the project-launch group over the past two weeks.
- A user who has been offline for 3 days opens the app — surface a concise catch-up summary of missed messages.
- Show me all images shared in the team chat related to the office renovation.

Requirement 1 (Basic)

- 1:1 messaging and group chat functionality with support for up to 100 participants per group.
- Support for media attachments including images, videos, and voice notes.
- Message delivery tracking with sent, delivered, and read status.
- Online/offline status and last-seen visibility.
- Offline message queueing — messages are queued for up to 30 days and delivered on reconnect.
- AI-enabled semantic search across historical conversations and media metadata.
- RAG-enabled conversational assistant for summarising past conversations.
- Hybrid search combining keyword and semantic retrieval.
- Tool-calling support for conversation retrieval and media lookup.
- Memory and session management for contextual conversation continuity.
- Notifications for messages and user activity.
- Input validation guardrails and basic content moderation.
- Simple front-end interface for messaging interactions.

Requirement 2 (Advanced)

- Multi-agent orchestration for moderation, summarisation, search, and delivery optimisation.
- Context-aware conversation summarisation across long chat histories with token optimisation for large-scale analysis.
- LLM-as-judge for validating generated summaries and contextual responses.
- DeepEval or equivalent evaluation framework for conversation summarisation quality; custom metrics for message delivery latency and retrieval relevance.

- Agent-to-agent communication for distributed message synchronisation and cross-group moderation intelligence.
- Root cause analysis for failed deliveries and synchronisation issues through collaborative AI agents.
- Feedback loop for improving search and summarisation quality over time.
- Smart notification agent that analyses user behaviour to determine optimal notification timing and reduce notification fatigue.
- Resilience testing for component failures and high-concurrency scenarios.
- Observability dashboards for real-time delivery rate monitoring, latency percentile tracking, and automated reporting.

Non-Functional Requirements

NFR Dimension	Target / Requirement
Latency	Message delivery < 500ms when both users are online (P99)
Scalability	500,000 messages/sec; 1 billion registered users; 50 million concurrent
Availability	99.99% uptime; resilient to individual component failures
Security	End-to-end encryption for all messages and media; forward secrecy
Data Retention	Messages not stored longer than necessary; configurable TTL
Reliability	Guaranteed at-least-once message delivery; deduplication on client
Observability	Real-time delivery monitoring; automated reporting; anomaly detection
Geo Distribution	Multi-region deployment; route to nearest region for latency optimisation
Compliance	GDPR right to deletion; data localisation requirements where applicable

Dataset

Dataset Name: Synthetic Messaging and Conversation Dataset

Format: CSV / JSON

Participants must create and use their own synthetic dataset simulating chat conversations, media exchanges, delivery events, user activity, and conversational summaries. A minimum of 100 users, 20 groups, and 50,000 messages is recommended.

Key Fields (minimum required schema):

Messages

Field Name	Type	Description
message_id	UUID	Unique message identifier
sender_id	UUID	User who sent the message
receiver_id	UUID	Recipient user ID (null for group messages)
group_id	UUID	Group identifier (null for direct messages)
timestamp	datetime	ISO 8601 send timestamp

<table>
<tr><td>message_content</td><td>string</td><td>Message text body</td></tr>
<tr><td>media_type</td><td>enum</td><td>text | image | voice | video | none</td></tr>
<tr><td>delivery_status</td><td>enum</td><td>sent | delivered | failed</td></tr>
<tr><td>read_status</td><td>enum</td><td>read | unread</td></tr>
<tr><td>conversation_summary</td><td>string</td><td>LLM-generated summary of the conversation thread</td></tr>
</table>

Users

<table>
<tr><th>Field Name</th><th>Type</th><th>Description</th></tr>
<tr><td>user_id</td><td>UUID</td><td>Unique user identifier</td></tr>
<tr><td>display_name</td><td>string</td><td>User display name</td></tr>
<tr><td>user_presence</td><td>enum</td><td>online | offline | last_seen_timestamp</td></tr>
<tr><td>status</td><td>enum</td><td>active | inactive | banned</td></tr>
<tr><td>registration_date</td><td>date</td><td>Account creation date</td></tr>
<tr><td>timezone</td><td>string</td><td>IANA timezone (e.g. Asia/Kolkata)</td></tr>
</table>

You may extend these schemas with message reactions, read-by lists for groups, or moderation flags. Document all extensions in your README.

Deliverables

1. Architecture Diagram

Include a high-level view of your system in JPEG or PDF. Show how data is processed from the synthetic dataset through ingestion, AI/ML layers, APIs, and the user-facing interface. Label all major components, data stores, and integration points.

2. Design

Articulate the system design decisions taken and the trade-offs made (e.g. choice of vector store, message broker, caching strategy, AI model selection, synchronous vs. asynchronous processing).

3. Full Executable Code (Microservice)

Focus on clarity and modularity. Provide a README explaining: (a) project setup — how to install dependencies and run the service; (b) sample usage — at least one end-to-end example showing a test input and the resulting output.

4. Panel Presentation (10 minutes)

Demo of your working solution outlining the design and approach (8 minutes). Q&A; (2 minutes).